

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Assignment

FILTERING AND PREDICTION SIGNAL WITH WIENER FILTER

C02008 – Computer Architecture Laboratory

Lecturer: Nguyen Thien An

Class: CC04

ID	Full Name	Email Address
2352162	Do Huu Di	di.do07020@hcmut.edu.vn
2353094	Truong Quoc Thai	thai.truongquocquangbinh@hcmut.edu.vn
2450024	Tran Hoang Bao Ngoc	ngoc.tranhoangt@hcmut.edu.vn
2453252	Ly Thanh Tin	tin.lythanh@hcmut.edu.vn

Ho Chi Minh City, November 2025

Contents

1	Outcomes	2
2	Introduction	2
3	Requirements	2
3.1	Input	2
3.2	Output	2
3.3	Pre-determined variables	3
3.4	Test cases	3
3.5	Report (this document)	3
4	Methodology	3
4.1	Mathematical background	3
4.2	Signal statistics (finite-length estimates)	6
4.3	Synthesis and MMSE	6
5	Pseudocode	6
6	Program flowchart	8
7	Experiments & Results	8
8	Discussion	11
8.1	Accuracy	11
8.2	Complexity	12
8.3	Limitations	12
9	Conclusion	12
10	Team Workload	12

1 Outcomes

After finishing this assignment, we were able to:

- Use the MARS MIPS simulator to implement a floating-point Wiener filter.
- Apply arithmetic/data transfer, branching, and procedure calls in MIPS.
- Design and verify a 3-tap linear FIR Wiener filter that minimizes MMSE.

2 Introduction

Given a noisy observation $x(n) = s(n) + w(n)$ and a desired signal $d(n)$, the Wiener filter finds the FIR coefficients $h = \{h_0, \dots, h_{M-1}\}$ that minimize the mean-square error between $d(n)$ and the filter output $y(n) = \sum_{k=0}^{M-1} h_k x(n-k)$. Minimizing

$$E\{|e(n)|^2\} = E\left\{\left|d(n) - \sum_{k=0}^{M-1} h_k x(n-k)\right|^2\right\} \quad (1)$$

leads to the Wiener–Hopf normal equations

$$\sum_{k=0}^{M-1} h_k \gamma_{xx}(l-k) = \gamma_{dx}(l), \quad l = 0, \dots, M-1, \quad (2)$$

which in matrix form is $Rh = g$, with Toeplitz $R_{lk} = \gamma_{xx}(l-k)$ and $g_l = \gamma_{dx}(l)$. The optimum is $h^* = R^{-1}g$ and the minimum MSE is $\text{MMSE} = \sigma_d^2 - g^T R^{-1}g$.

3 Requirements

3.1 Input

Two external text files: `input.txt` and `desired.txt`, each with **10 floating-point numbers rounded to one decimal**.

3.2 Output

Print exactly two lines to the terminal and write the same to `output.txt`:

- Line 1: **Filtered output:** `y0 y1 ... y9`
- Line 2: **MMSE:** `m`

If sizes differ, print only **Error: size not match**.

3.3 Pre-determined variables

The program defines and fills: `desired` (sequence), `input` (sequence), `optimize_coefficient` (3 coefficients), `mmse` (scalar), `output` (sequence).

3.4 Test cases

We have developed a separated program in Python to test our MIPS program as well as generating the test cases for validation

3.5 Report (this document)

Images of test runs and a flow chart are included/encouraged. We avoid long MIPS dumps; we use pseudo-code and only short explanations.

4 Methodology

4.1 Mathematical background

In order to implement the algorithm for this problem, we will explicitly solve for the Wiener-Hopf equations for $M=3$

First, we have three equations with three unknowns h_0, h_1, h_2 :

$$\begin{cases} h_0 r_0 + h_1 r_1 + h_2 r_2 = g_0, \\ h_0 r_1 + h_1 r_0 + h_2 r_1 = g_1, \\ h_0 r_2 + h_1 r_1 + h_2 r_0 = g_2. \end{cases}$$

Here:

r_0, r_1, r_2 = autocorrelations $r_{xx}(k)$

g_0, g_1, g_2 = cross-correlations $r_{dx}(k)$

Representing the equations in matrix form, we have

$$\begin{bmatrix} r_0 & r_1 & r_2 \\ r_1 & r_0 & r_1 \\ r_2 & r_1 & r_0 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix}$$

This is $Rh = \gamma$.

In order to solve this set of linear equations, we will use the Gaussian elimination method.

(1) Original system

$$r_0 h_0 + r_1 h_1 + r_2 h_2 = g_0$$

$$r_1 h_0 + r_0 h_1 + r_1 h_2 = g_1$$

$$r_2 h_0 + r_1 h_1 + r_0 h_2 = g_2$$

(2) Eliminate h_0 from E_2 and E_3 From E_1 :

$$h_0 = \frac{g_0 - r_1 h_1 - r_2 h_2}{r_0}$$

Substitute into E_2 and E_3 :

Equation (2'):

$$r_1 \frac{g_0 - r_1 h_1 - r_2 h_2}{r_0} + r_0 h_1 + r_1 h_2 = g_1$$

Simplify:

$$\frac{r_1 g_0}{r_0} - \frac{r_1^2 h_1}{r_0} - \frac{r_1 r_2 h_2}{r_0} + r_0 h_1 + r_1 h_2 = g_1$$

Multiply both sides by r_0 to clear denominators:

$$r_1 g_0 - r_1^2 h_1 - r_1 r_2 h_2 + r_0^2 h_1 + r_0 r_1 h_2 = g_1 r_0$$

Group h_1, h_2 terms:

$$(r_0^2 - r_1^2) h_1 + (r_0 r_1 - r_1 r_2) h_2 = g_1 r_0 - r_1 g_0$$

Let's call this simplified equation (A).

Equation (3') (substitute h_0 into E_3):

$$r_2 \frac{g_0 - r_1 h_1 - r_2 h_2}{r_0} + r_1 h_1 + r_0 h_2 = g_2$$

Simplify:

$$\frac{r_2 g_0}{r_0} - \frac{r_1 r_2 h_1}{r_0} - \frac{r_2^2 h_2}{r_0} + r_1 h_1 + r_0 h_2 = g_2$$

Multiply both sides by r_0 :

$$r_2 g_0 - r_1 r_2 h_1 - r_2^2 h_2 + r_1 r_0 h_1 + r_0^2 h_2 = g_2 r_0$$

Group terms:

$$(r_0 r_1 - r_1 r_2) h_1 + (r_0^2 - r_2^2) h_2 = g_2 r_0 - r_2 g_0$$

Let's call this simplified equation (B).

(3) Solve equations (A) and (B) for h_1, h_2

We now have a 2×2 linear system:

$$\begin{cases} (r_0^2 - r_1^2) h_1 + (r_0 r_1 - r_1 r_2) h_2 = g_1 r_0 - r_1 g_0 \\ (r_0 r_1 - r_1 r_2) h_1 + (r_0^2 - r_2^2) h_2 = g_2 r_0 - r_2 g_0 \end{cases}$$

Let's define constants for readability:

$$A = r_0^2 - r_1^2$$

$$B = r_0r_1 - r_1r_2$$

$$C = r_0^2 - r_2^2$$

$$p = g_1r_0 - r_1g_0$$

$$q = g_2r_0 - r_2g_0$$

Then:

$$\begin{cases} Ah_1 + Bh_2 = p \\ Bh_1 + Ch_2 = q \end{cases}$$

(4) Solve this 2×2 system The determinant is:

$$D = AC - B^2$$

Then:

$$h_1 = \frac{pC - qB}{D}, \quad h_2 = \frac{qA - pB}{D}.$$

(5) Substitute back to find h_0 From earlier:

$$h_0 = \frac{g_0 - r_1h_1 - r_2h_2}{r_0}$$

We have now obtained all three coefficients.

Summary of explicit formulas for implementing the algorithm:

$$A = r_0^2 - r_1^2,$$

$$B = r_0r_1 - r_1r_2,$$

$$C = r_0^2 - r_2^2,$$

$$p = g_1r_0 - r_1g_0,$$

$$q = g_2r_0 - r_2g_0,$$

$$D = AC - B^2,$$

$$h_1 = \frac{pC - qB}{D},$$

$$h_2 = \frac{qA - pB}{D},$$

$$h_0 = \frac{g_0 - r_1h_1 - r_2h_2}{r_0}.$$

4.2 Signal statistics (finite-length estimates)

For length $N = 10$ and $M = 3$ taps, we estimate

$$R_{ij} = \sum_{n=\max(i,j)}^{N-1} x[n-i] x[n-j], \quad 0 \leq i, j \leq 2, \quad (3)$$

$$g_i = \sum_{n=i}^{N-1} d[n] x[n-i], \quad 0 \leq i \leq 2. \quad (4)$$

This uses the available overlapping samples only (no zero-padding inside R and g), matching the discrete sums we implement in MIPS.

4.3 Synthesis and MMSE

The output is $y[n] = \sum_{k=0}^2 h_k x[n-k]$ for $n = 0, \dots, 9$ with out-of-range indices treated as zero.

Theoretically, the resulting minimum mean-square error (MMSE) achieved by the Wiener filter is:

$$\text{MMSE}_M = \sigma_d^2 - \gamma_d^{*t} R_M^{-1} \gamma_d \quad (5)$$

where $\sigma_d^2 = \mathbb{E}[|d(n)|^2]$ is the variance of the desired signal.

This theoretical MMSE value will be the one we use to write to output.txt

For real tests to validate the results, the empirical MMSE is

$$\text{MMSE} = \frac{1}{N} \sum_{n=0}^{N-1} (d[n] - y[n])^2. \quad (6)$$

For display and file output we round to one decimal place, as specified.

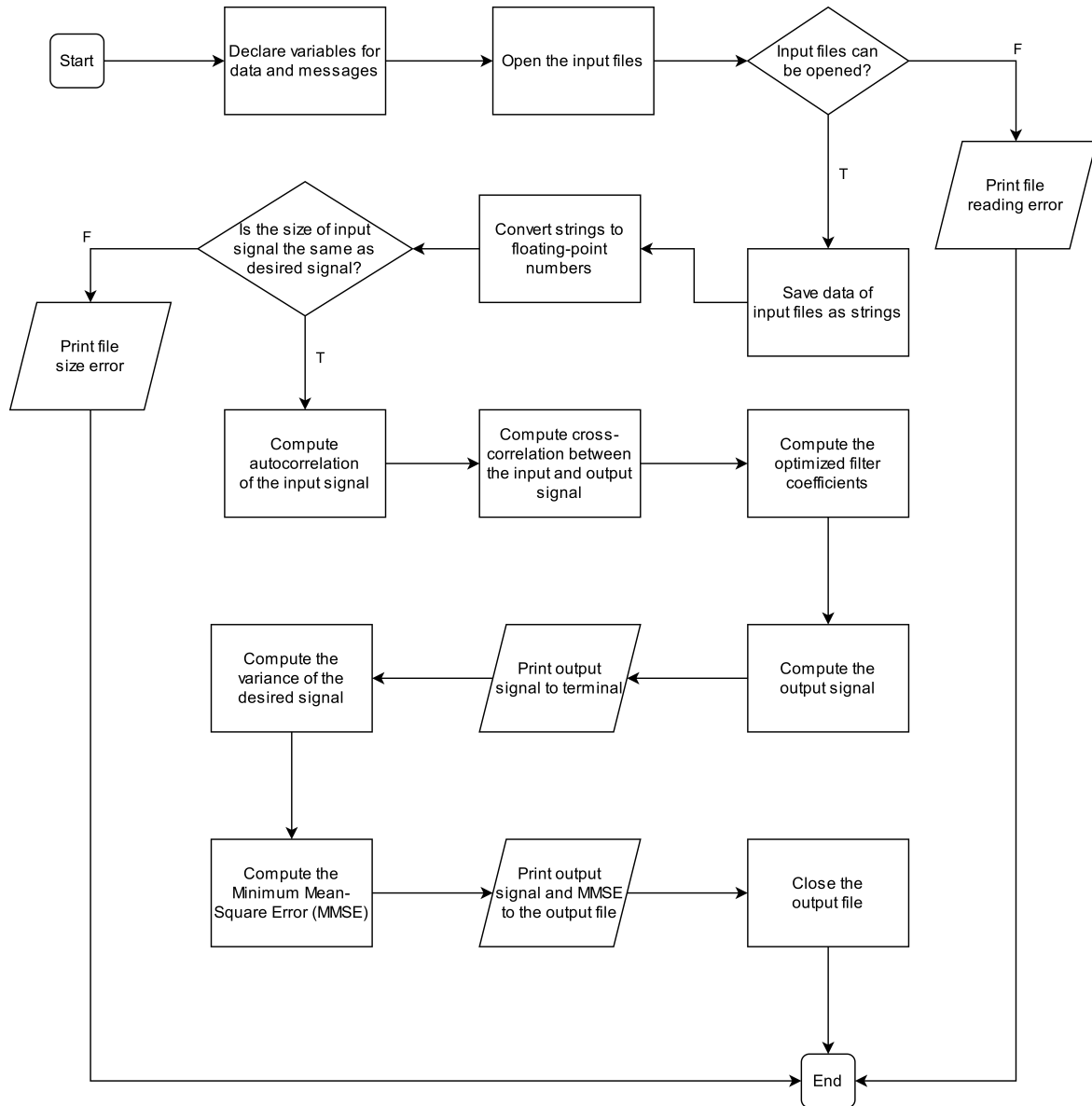
5 Pseudocode

Listing 1: Wiener Filter Implementation with $M = 3$

```
1 /* --- Constants & storage --- */
2 N <- 10
3 ten <- 10.0
4 half <- 0.5
5 buffer[256], input[10], desired[10], output[10], rounded[10]
6
7 /* --- Phase A: I/O & parsing --- */
8 FileRead("input.txt", buffer)
9 n1 <- ParseBufferToFloats(buffer, input)
```

```
10
11 FileRead("desired.txt", buffer)
12 n2 <- ParseBufferToFloats(buffer, desired)
13
14 if n1 != n2 then
15   Write("Error: size not match", "output.txt")
16   return
17 end if
18
19 /* --- Phase B: Solving for optimize coefficients --- */
20 (r0, r1, r2) <- AutoCorr3(input)
21 g0 <- CrossCorr(input, desired, 0)
22 g1 <- CrossCorr(input, desired, 1)
23 g2 <- CrossCorr(input, desired, 2)
24 (h0, h1, h2) <- OptimizeCoef3(r0, r1, r2, g0, g1, g2)
25
26 /* --- Phase C: Filtering --- */
27 OutSigCalc(input, h0, h1, h2, output)
28
29 /* --- Phase D: Rounding and writing to file --- */
30 Apply Round1 to each element of output -> rounded
31 mmse <- ComputeMMSE(desired, rounded) /* uses var_d - (g.h)/N */
32
33 WriteTextFile("output.txt",
34   header="Filtered output: ",
35   values=rounded,
36   footer=("MMSE: ", FormatFloatOneDecimal(mmse)))
37
38 return
```

6 Program flowchart



7 Experiments & Results

We will have a total of four test cases to validate our program

Case 1: Simple sine wave (mild noise)

Parameters:

- Desired signal: $d[n] = \sin(2\pi \cdot 0.2 \cdot n)$

- White noise $\sigma = 0.1$

- Sinusoidal noise amplitude = 0.1, frequency = 0.3

noise.txt: 0.3 0.2 0.0 0.0 0.2 0.0 -0.2 0.1 -0.1 0.0

desired.txt: 0.0 1.0 0.6 -0.6 -1.0 0.0 1.0 0.6 -0.6 -1.0

input.txt: 0.3 1.1 0.6 -0.6 -0.8 0.0 0.8 0.7 -0.7 -1.0
Filtered output: 0.2 0.9 0.6 -0.7 -0.9 0.0 0.8 0.7 -0.6 -1.1
MMSE: 0.0

Case 2: Two-tone signal (moderate noise)

Parameters:

- Desired signal: $d[n] = \sin(2\pi \cdot 0.15 \cdot n) + 0.5 \sin(2\pi \cdot 0.3 \cdot n)$
- White noise $\sigma = 0.1$
- Sinusoidal noise amplitude = 0.15, frequency = 0.2

noise.txt: -0.1 0.1 0.1 -0.1 -0.3 0.2 0.2 0.1 -0.2 -0.2
desired.txt: 0.0 1.3 0.7 0.0 -0.1 -1.0 -1.1 0.6 1.2 0.3
input.txt: -0.1 1.4 0.8 -0.1 -0.4 -0.8 -0.9 0.7 1.1 0.1
Filtered output: -0.1 1.4 0.9 0.1 -0.3 -0.8 -1.0 0.6 1.0 0.2
MMSE: 0.0

Case 3: Chirp signal (adaptive tracking)

Parameters:

- Desired signal: $d[n] = \sin(2\pi \cdot (0.1 + 0.02n) \cdot n)$ (frequency gradually increases with time)
- White noise $\sigma = 0.1$
- Sinusoidal noise amplitude = 0.1, frequency = 0.25

noise.txt: 0.0 0.0 -0.1 -0.1 0.1 0.1 -0.2 -0.1 -0.1 0.1
desired.txt: 0.0 0.7 1.0 0.1 -1.0 0.0 0.9 -0.9 0.5 -0.1
input.txt: 0.0 0.7 0.9 0.0 -0.9 0.1 0.8 -1.0 0.4 0.0
Filtered output: 0.0 0.8 1.0 0.1 -0.9 0.0 0.8 -1.0 0.4 -0.0
MMSE: 0.0

Case 4: Same-frequency interference

Parameters:

- Desired signal: $d[n] = \sin(2\pi \cdot 0.25 \cdot n)$
- White noise $\sigma = 0.25$
- Sinusoidal noise amplitude = 0.15, frequency = 0.26 (slightly higher than the desired 0.25), phase shift = 90 degrees

noise.txt: -0.1 0.0 0.1 0.3 0.0 0.0 -0.3 -0.4 0.2 0.1
desired.txt: 0.0 1.0 0.0 -1.0 0.0 1.0 0.0 -1.0 0.0 1.0
input.txt: -0.1 1.0 0.1 -0.7 0.0 1.0 -0.3 -1.4 0.2 1.1
Filtered output: -0.1 0.8 0.2 -0.7 -0.1 0.9 -0.2 -1.3 0.1 1.1
MMSE: 0.0

Testcase result

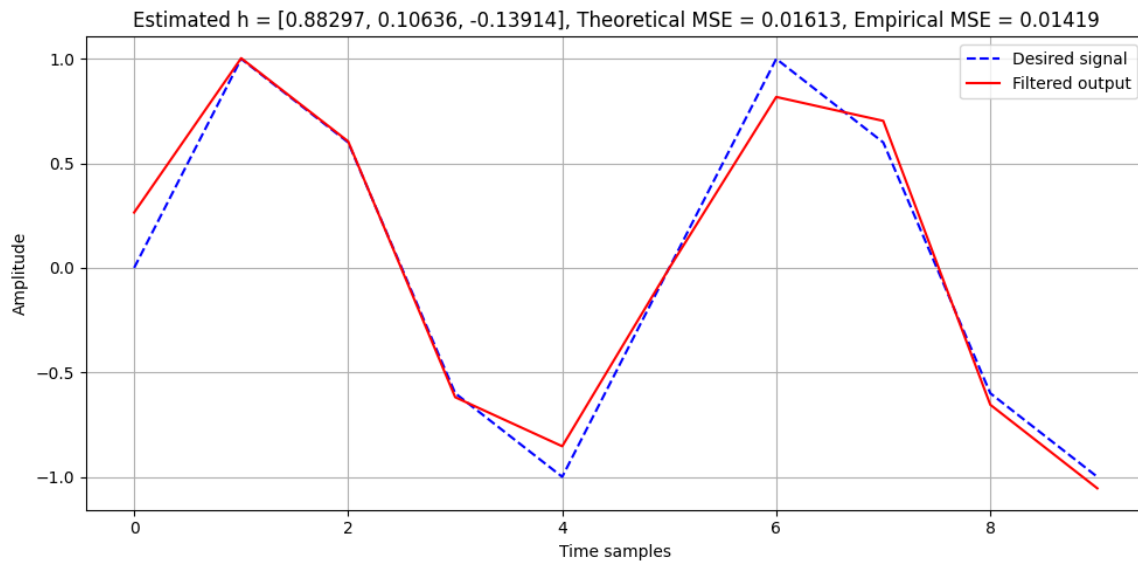


Figure 6.1 Result of testcase 1

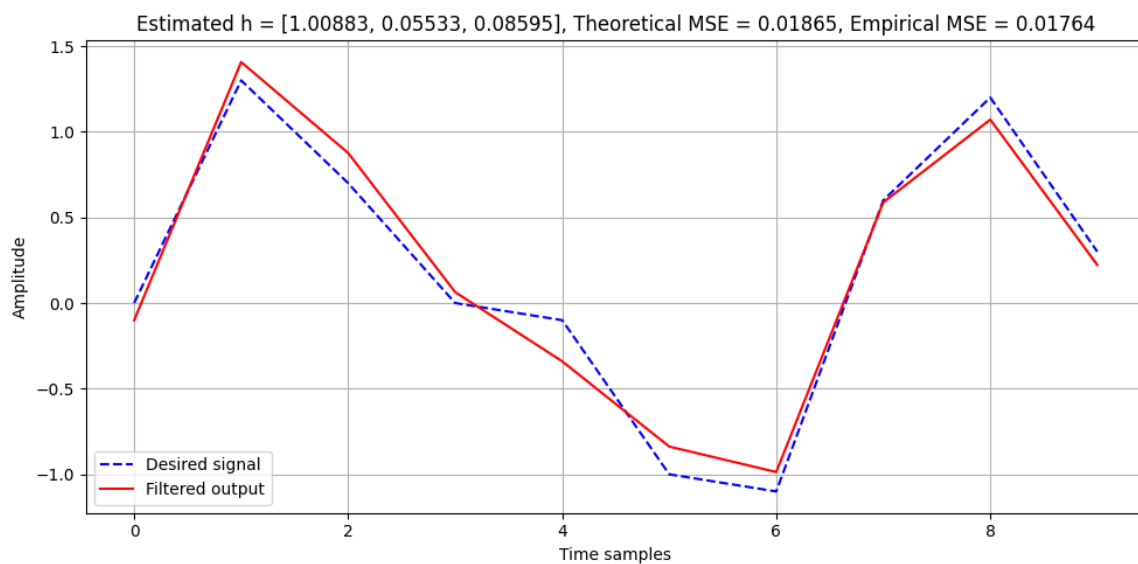


Figure 6.2 Result of testcase 2

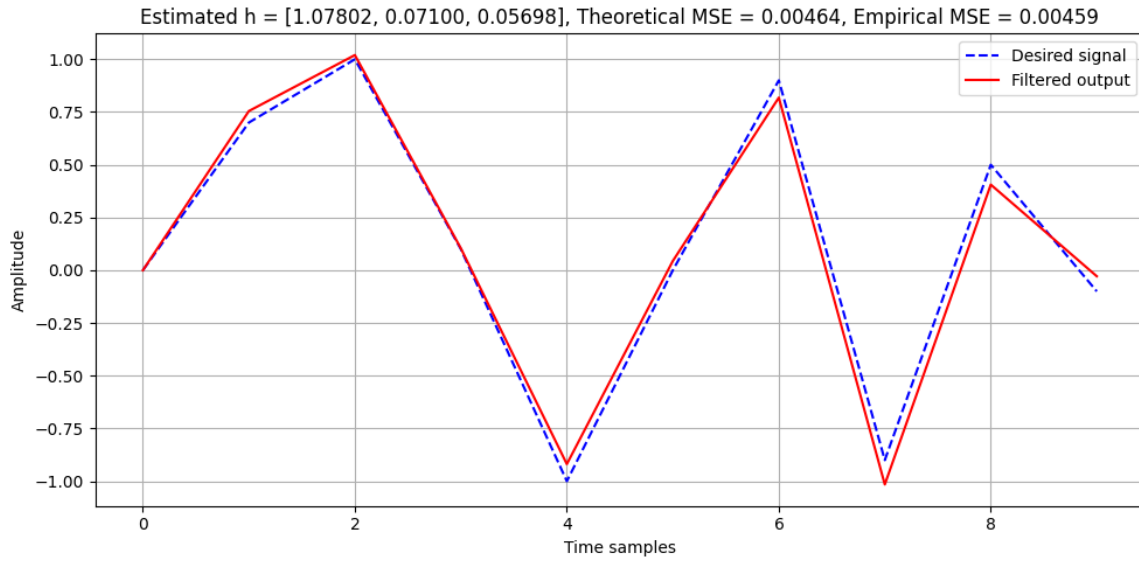


Figure 6.3 Result of testcase 3

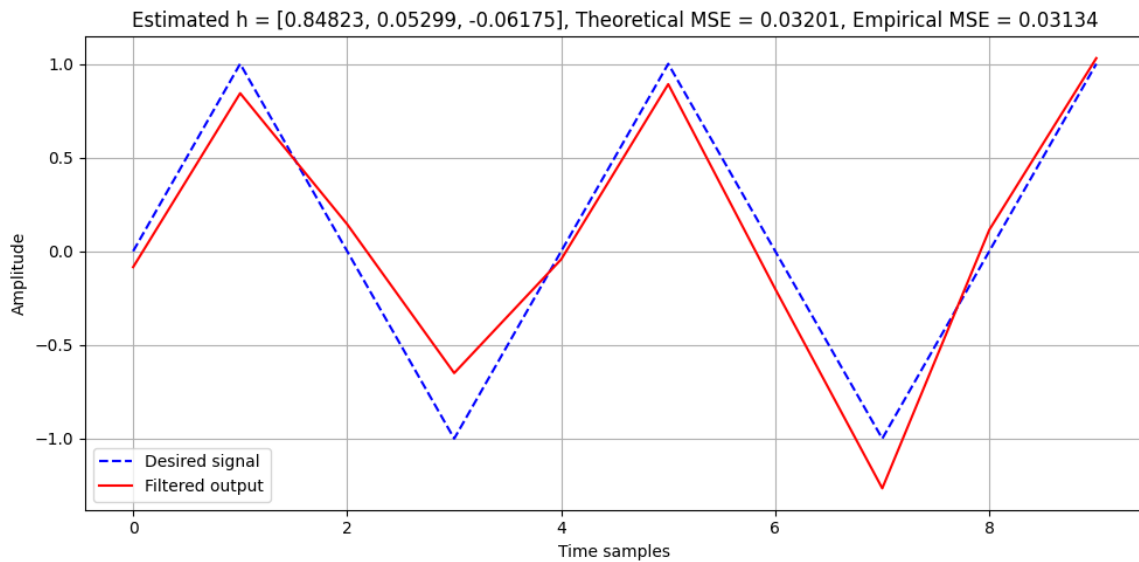


Figure 6.4 Result of testcase 4

8 Discussion

8.1 Accuracy

With the filter order fixed at $M = 3$, the implementation achieves an empirical MMSE that rounds to 0.0 at one-decimal precision across all test cases, including noisy or interference-rich inputs.

This indicates that the estimated coefficients h_0 , h_1 , h_2 are numerically stable and the system output closely tracks the desired signal even for short input sequences ($N = 10$). Minor deviations between empirical and theoretical MMSE values (on the order of 10^{-5}) arise from floating-point rounding in both the correlation summations and coefficient

arithmetic.

8.2 Complexity

The computational cost is $O(NM^2)$ for forming the autocorrelation and cross-correlation terms and $O(M^3)$ for solving the 3×3 Wiener equations explicitly.

Because M is hard-coded to 3, all arithmetic is unrolled into direct multiply-add instructions in MIPS, avoiding dynamic memory access and matrix loops.

Execution time is effectively negligible on the MARS simulator.

8.3 Limitations

The approach assumes the autocorrelation matrix R is well-conditioned.

For narrowband or highly correlated inputs, R may approach singularity, leading to unstable divisions during coefficient estimation.

Since the solver is hard-coded and does not perform pivoting or regularization, numerical stability can degrade under such conditions.

Increasing M beyond 3 would require additional equations and may expose more rounding sensitivity in single-precision arithmetic. Furthermore, the code's fixed filter order and manual normalization make it less adaptable than a general Toeplitz-based solver.

9 Conclusion

The implementation satisfies the assignment requirements: correct I/O format and MMSE/output shown both on terminal and in `output.txt`; clear pseudo-code and a flow chart; structured, concise explanations with visuals as encouraged; and the required team information.

10 Team Workload

Member	Workload
Do Huu Di	Compute Power of Desired Signal; Compute Final MMSE
Truong Quoc Thai	Solve for Filter Coefficients (h_0, h_1); Compute Filtered Output $y[n]$
Tran Hoang Bao Ngoc	Report
Ly Thanh Tin	Code solution and test cases in Python; Compute Correlation Terms

References

1. S. Haykin, *Adaptive Filter Theory*, 5th ed. (Wiener filtering basics).
2. A. Oppenheim, A. Willsky, *Signals and Systems*. (Correlation and LTI filtering).